THERMO PROPERTY API

Integration Guide

How to call the API, with the request, response and errors of every endpoint

Version	2.3.0
Date	23 October 2026
Base URL	http://192.168.11.216:4300/api
Interactive documentation	http://192.168.11.216:4300/api/docs
Format	HTTP and JSON (UTF-8)
Access	Company network or VPN
Authentication	None in the current deployment

Contents

Section	Title	Endpoint
1	Overview	
2	Request and response format	
3	Errors	
4	Heat capacity	POST /cp_calc
5	Density	POST /density_calc
6	Molecular weight	POST /mw_calc
7	Vapor fraction	POST /vf_calc
8	Heat capacity: vapor, aqueous, organic	POST /triphase_cp_calc
9	Molecular weight: vapor, aqueous, organic	POST /triphase_mw_calc
10	Phase fractions: vapor, aqueous, organic	POST /triphase_vf_calc
11	Stream properties	POST /eos_stream_calc
12	Enthalpy and heat duty	POST /eos_enthalpy_calc
13	Flow versus valve opening	POST /vent_flow_calc
14	Health check	GET /health
15	Client examples and version history	

1. Overview

The Thermo Property API calculates thermodynamic properties of mixtures. It has four groups. Every endpoint uses the same style of request and returns the same style of response, so once one works, the others follow.

Group	Sections	Use it when
Property APIs	4 to 10	You need one specific property (heat capacity, density, molecular weight, vapor fraction, or the vapor / aqueous / organic split). Vapor–liquid model, one endpoint per property.
EOS Stream API	11	You need the complete stream result in one call, or the phase behaviour matters (for example water with hydrocarbons). Peng-Robinson equation of state, three-phase flash.
Enthalpy API	12	You need the enthalpy of a stream at an inlet and an outlet condition and the heat duty between them.
Vent API	13	You have measured flow at several valve openings and need the flow at other openings. Works on your own data points.

Endpoints

Method and path	Section	Returns
POST /cp_calc	4	Heat capacity
POST /density_calc	5	Density
POST /mw_calc	6	Molecular weight
POST /vf_calc	7	Vapor fraction
POST /triphase_cp_calc	8	Heat capacity: vapor, aqueous, organic
POST /triphase_mw_calc	9	Molecular weight: vapor, aqueous, organic
POST /triphase_vf_calc	10	Phase fractions: vapor, aqueous, organic
POST /eos_stream_calc	11	Stream properties
POST /eos_enthalpy_calc	12	Enthalpy and heat duty
POST /vent_flow_calc	13	Flow versus valve opening
GET /health	14	Service status and version

Rules that apply everywhere

- All calculations use **POST** with a JSON body and the header `Content-Type: application/json`. `/health` uses GET.
- Temperature in **K**, pressure in **Pa**. Components are chemical names or CAS numbers. Mole fractions may also be given as mole percent; they are rescaled to add up to 1.
- A phase that does not exist at the given conditions is returned as **null**, never 0, and this is not an error.
- Values come back at full precision; round them in your own software. Unknown request fields are ignored.
- Every response has a `units` object that states the unit of each returned value, and a `request_id` for support.
- A calculation that takes longer than 30 seconds is stopped (error `calculation_timeout`).

2. Request and response format

Common request (all calculations except sections 12 and 13)

Field	Type	Rules
`T`	number	Temperature in K. Must be above 0.
`P`	number	Pressure in Pa. Must be above 0 (1 atm = 101325 Pa).
`components`	array of text	Names or CAS numbers, for example `"water"` or `"7732-18-5"`. At least one, no duplicates, no limit on the number.
`zs`	array of numbers	Mole fractions in the same order and length as `components`. Not negative; the sum must be above 0. Rescaled to add up to 1; entries of 0 are allowed.

```
{ "T": 300, "P": 101325, "components": ["water", "hexane"], "zs": [0.5, 0.5] }
```

Successful response (HTTP 200)

Every success has the same four parts. The example is the response of section 4.

```
{
  "status": "success",
  "data": {
    "Cp_liquid": 135.909,
    "Cp_vapor": null,
    "Cp_mix": 135.909,
    "Vapor_fraction": 0.0,
    "Liquid_fraction": 1.0
  },
  "units": {
    "Cp_liquid": "J/mol/K",
    "Cp_vapor": "J/mol/K",
    "Cp_mix": "J/mol/K",
    "Vapor_fraction": "mole fraction",
    "Liquid_fraction": "mole fraction"
  },
  "meta": { "request_id": "a1b2c3d4e5f6", "elapsed_ms": 14.2, "zs_normalized": false }
}
```

Part	Meaning
`status`	Always `"success"`.
`data`	The results; their fields are listed in each endpoint section.
`units`	Unit of each value in `data`.
`meta`	`request_id` (quote it when you contact support), `elapsed_ms` (server time) and, for calculations with a composition, `zs_normalized` (true if your fractions were rescaled).

Error response (HTTP 4xx or 5xx)

```
{
  "status": "error",
  "error": {
    "code": "validation_error",
    "message": "Request body is invalid.",
    "details": [ { "field": "T", "message": "Input should be greater than 0" } ]
  },
  "request_id": "a1b2c3d4e5f6"
}
```

Use `error.code` in your software; it never changes. `message` is for people, and `details` carries the specifics (a list of fields for `validation_error`).

3. Errors

Always check the HTTP status first: 200 is success, anything else is an error in the format shown in section 2.

Error code	HTTP	Cause	What to do
`validation_error`	422	The request is invalid: bad JSON, wrong type, T or P not above 0, lists of different length, negative fractions, duplicate or empty component, too few data points.	Read `error.details` and fix the listed fields.
`unknown_component`	422	A name or CAS number is not known. `details.component` gives it.	Use another spelling or the CAS number.
`flash_failed`	422	The equilibrium calculation failed for this state.	Check temperature, pressure and composition.
`property_unavailable`	422	Data needed for a component is missing. `details.components` lists them.	Remove or replace the component.
`water_required`	422	A triphase endpoint (sections 8 to 10) was called without water.	Add water, or use the EOS Stream API.
`fit_failed`	422	Vent API only: too few different opening values for the degree.	Provide more different openings or lower the degree.
`unauthorized`	401	An API key is enabled on the server and the `X-API-Key` header is missing or wrong.	Send the correct key.
`not_found`	404	The URL does not exist.	Check the path, including `/api`.
`method_not_allowed`	405	Wrong HTTP method, for example GET on a calculation.	Use POST.
`calculation_failed`	500	The model could not be built.	Report it with the `request_id`.
`internal_error`	500	Unexpected server error.	Report it with the `request_id`.
`calculation_timeout`	504	The calculation took longer than 30 seconds.	Simplify the input, or contact support.

Handling errors in your software

- Decide by HTTP status and `error.code`, never by the text of `message`.
- Do not repeat a 422 request unchanged: the input has to change. A 504 may be repeated once with simpler input.
- Log the `request_id` of every failure; it lets support find the request in the server log.

4. Heat capacity

POST /cp_calc

http://192.168.11.216:4300/api/cp_calc

Heat capacity of the liquid phase, the vapor phase and the whole mixture, together with the phase fractions, from a vapor–liquid flash at the given T and P.

Request

The request is the common one described in section 2: `T` (K), `P` (Pa), `components` and `zs`.

```
{ "T": 300, "P": 101325, "components": ["water", "hexane"], "zs": [0.5, 0.5] }
```

Example mixture: water 50 mol % and hexane 50 mol % at 300 K and 101325 Pa. At this state the model finds a single liquid, so vapor values are null.

Response

The example is the `data` part of the response; `units` and `meta` are as in section 2.

Field	Type	Unit	Description
`Cp_liquid`	number or null	J/mol/K	Liquid-phase heat capacity. null if there is no liquid.
`Cp_vapor`	number or null	J/mol/K	Vapor-phase heat capacity (ideal-gas). null if there is no vapor.
`Cp_mix`	number	J/mol/K	Fraction-weighted mixture heat capacity.
`Vapor_fraction`	number	mole fraction	Vapor fraction (0 to 1).
`Liquid_fraction`	number	mole fraction	Liquid fraction (1 − vapor fraction).

```
{
  "Cp_liquid": 135.909,
  "Cp_vapor": null,
  "Cp_mix": 135.909,
  "Vapor_fraction": 0.0,
  "Liquid_fraction": 1.0
}
```

Errors

`validation_error` (422), `unknown_component` (422), `flash_failed` (422), `property_unavailable` (422), `calculation_timeout` (504). Section 3 explains each code and what to do.

Notes

- A phase that does not exist is null, never 0.

5. Density

POST /density_calc

http://192.168.11.216:4300/api/density_calc

Density of the mixture at the given T and P.

Request

The request is the common one described in section 2: `T` (K), `P` (Pa), `components` and `zs`.

```
{ "T": 300, "P": 101325, "components": ["water", "hexane"], "zs": [0.5, 0.5] }
```

Example mixture: water 50 mol % and hexane 50 mol % at 300 K and 101325 Pa. At this state the model finds a single liquid, so vapor values are null.

Response

The example is the `data` part of the response; `units` and `meta` are as in section 2.

Field	Type	Unit	Description
`rho_mix`	number	kg/m³	Mixture density.
`phase`	string or null	–	Phase state: `g` gas, `l` liquid, `l/g` two-phase.

```
{ "rho_mix": 694.6499, "phase": "l" }
```

Errors

`validation_error` (422), `unknown_component` (422), `property_unavailable` (422), `calculation_timeout` (504). Section 3 explains each code and what to do.

Notes

- For methane/ethane 50/50 at 423.15 K and 29411.81 Pa the result is 0.1927403 kg/m³ with phase `g`.

6. Molecular weight

POST /mw_calc

http://192.168.11.216:4300/api/mw_calc

Molecular weight of the liquid phase, the vapor phase and the whole mixture.

Request

The request is the common one described in section 2: `T` (K), `P` (Pa), `components` and `zs`.

```
{ "T": 300, "P": 101325, "components": ["water", "hexane"], "zs": [0.5, 0.5] }
```

Example mixture: water 50 mol % and hexane 50 mol % at 300 K and 101325 Pa. At this state the model finds a single liquid, so vapor values are null.

Response

The example is the `data` part of the response; `units` and `meta` are as in section 2.

Field	Type	Unit	Description
`MW_liquid`	number or null	g/mol	Liquid phase. null if there is no liquid.
`MW_vapor`	number or null	g/mol	Vapor phase. null if there is no vapor.
`MW_avg`	number	g/mol	Overall mixture.

```
{ "MW_liquid": 52.09532, "MW_vapor": null, "MW_avg": 52.09532 }
```

Errors

`validation_error` (422), `unknown_component` (422), `flash_failed` (422), `property_unavailable` (422), `calculation_timeout` (504). Section 3 explains each code and what to do.

7. Vapor fraction

POST /vf_calc

http://192.168.11.216:4300/api/vf_calc

How much of the mixture is vapor and how much is liquid at the given T and P.

Request

The request is the common one described in section 2: `T` (K), `P` (Pa), `components` and `zs`.

```
{ "T": 300, "P": 101325, "components": ["water", "hexane"], "zs": [0.5, 0.5] }
```

Example mixture: water 50 mol % and hexane 50 mol % at 300 K and 101325 Pa. At this state the model finds a single liquid, so vapor values are null.

Response

The example is the `data` part of the response; `units` and `meta` are as in section 2.

Field	Type	Unit	Description
`Vapor_fraction`	number	mole fraction	0 = all liquid, 1 = all vapor.
`Liquid_fraction`	number	mole fraction	1 − vapor fraction.

```
{ "Vapor_fraction": 0.0, "Liquid_fraction": 1.0 }
```

Errors

`validation_error` (422), `unknown_component` (422), `flash_failed` (422), `calculation_timeout` (504). Section 3 explains each code and what to do.

Notes

- Mole fractions that do not add up to 1 are rescaled; `meta.zs_normalized` is then true.

8. Heat capacity: vapor, aqueous, organic

POST /triphase_cp_calc

http://192.168.11.216:4300/api/triphase_cp_calc

Heat capacity of the vapor, aqueous (water) and organic phases of a mixture that contains water.

Request

The request is the common one described in section 2: `T` (K), `P` (Pa), `components` and `zs`. **The mixture must contain water.**

```
{ "T": 300, "P": 101325, "components": ["water", "hexane"], "zs": [0.5, 0.5] }
```

Example mixture: water 50 mol % and hexane 50 mol % at 300 K and 101325 Pa. At this state the model finds a single liquid, so vapor values are null.

Response

The example is the `data` part of the response; `units` and `meta` are as in section 2.

Field	Type	Unit	Description
`Cp_aqueous`	number or null	J/mol/K	null if there is no liquid water.
`Cp_organic`	number or null	J/mol/K	null if there is no organic liquid.
`Cp_vapor`	number or null	J/mol/K	null if there is no vapor.
`Cp_mix`	number	J/mol/K	Weighted mixture heat capacity.

```
{ "Cp_aqueous": 75.31529, "Cp_organic": 196.5026, "Cp_vapor": null, "Cp_mix": 135.909 }
```

Errors

`validation_error` (422), `unknown_component` (422), `water_required` (422), `flash_failed` (422), `calculation_timeout` (504). Section 3 explains each code and what to do.

- `water_required`: The mixture does not contain water. Add `"water"` or `"7732-18-5"`, or use the EOS Stream API.

```
{
  "status": "error",
  "error": {
    "code": "water_required",
    "message": "Triphase endpoints require water in the mixture (use 'water' or '7732-18-5').",
    "details": null
  },
  "request_id": "a1b2c3d4e5f6"
}
```

Notes

- The liquid is split by approximation: **aqueous** = water, **organic** = all other components. This suits water and hydrocarbons that do not mix; for a rigorous split use the EOS Stream API.

9. Molecular weight: vapor, aqueous, organic

POST /triphase_mw_calc

http://192.168.11.216:4300/api/triphase_mw_calc

Molecular weight of the vapor, aqueous and organic phases of a mixture that contains water.

Request

The request is the common one described in section 2: `T` (K), `P` (Pa), `components` and `zs`. **The mixture must contain water.**

```
{ "T": 300, "P": 101325, "components": ["water", "hexane"], "zs": [0.5, 0.5] }
```

Example mixture: water 50 mol % and hexane 50 mol % at 300 K and 101325 Pa. At this state the model finds a single liquid, so vapor values are null.

Response

The example is the `data` part of the response; `units` and `meta` are as in section 2.

Field	Type	Unit	Description
`MW_aqueous`	number or null	g/mol	Water. null if there is no liquid water.
`MW_organic`	number or null	g/mol	null if there is no organic liquid.
`MW_vapor`	number or null	g/mol	null if there is no vapor.
`MW_mix`	number	g/mol	Overall mixture.

```
{ "MW_aqueous": 18.01528, "MW_organic": 86.17536, "MW_vapor": null, "MW_mix": 52.09532 }
```

Errors

`validation_error` (422), `unknown_component` (422), `water_required` (422), `flash_failed` (422), `calculation_timeout` (504). Section 3 explains each code and what to do.

- `water_required`: The mixture does not contain water. Add `"water"` or `"7732-18-5"`, or use the EOS Stream API.

Notes

- Same aqueous / organic split as section 8.

10. Phase fractions: vapor, aqueous, organic

POST /triphase_vf_calc

http://192.168.11.216:4300/api/triphase_vf_calc

How the total mixture divides into vapor, aqueous and organic phases.

Request

The request is the common one described in section 2: `T` (K), `P` (Pa), `components` and `zs`. **The mixture must contain water.**

```
{ "T": 300, "P": 101325, "components": ["water", "hexane"], "zs": [0.5, 0.5] }
```

Example mixture: water 50 mol % and hexane 50 mol % at 300 K and 101325 Pa. At this state the model finds a single liquid, so vapor values are null.

Response

The example is the `data` part of the response; `units` and `meta` are as in section 2.

Field	Type	Unit	Description
`vapor_fraction`	number	mole fraction	Share of the total mixture that is vapor.
`aqueous_fraction`	number	mole fraction	Share of the total mixture that is aqueous liquid.
`organic_fraction`	number	mole fraction	Share of the total mixture that is organic liquid.
`liquid_fraction`	number	mole fraction	Aqueous plus organic.
`water_in_liquid`	number	mole fraction	Water share within the liquid.
`organic_in_liquid`	number	mole fraction	Non-water share within the liquid.

```
{
  "vapor_fraction": 0.0,
  "aqueous_fraction": 0.5,
  "organic_fraction": 0.5,
  "liquid_fraction": 1.0,
  "water_in_liquid": 0.5,
  "organic_in_liquid": 0.5
}
```

Errors

`validation_error` (422), `unknown_component` (422), `water_required` (422), `flash_failed` (422), `calculation_timeout` (504). Section 3 explains each code and what to do.

- `water_required`: The mixture does not contain water. Add `"water"` or `"7732-18-5"`, or use the EOS Stream API.

Notes

- vapor + aqueous + organic fractions add up to 1.
- Same aqueous / organic split as section 8.

11. Stream properties

POST /eos_stream_calc

http://192.168.11.216:4300/api/eos_stream_calc

The complete result for a stream in one call, from a Peng-Robinson equation of state with a three-phase flash: the phase situation, and for every phase that exists its fraction, molecular weight, heat capacity and density, plus the mixture values. Water is optional.

Request

The request is the common one described in section 2: `T` (K), `P` (Pa), `components` and `zs`. Components with a mole fraction of 0 are skipped.

```
{ "T": 300, "P": 101325, "components": ["water", "hexane"], "zs": [0.5, 0.5] }
```

Example mixture: water 50 mol % and hexane 50 mol % at 300 K and 101325 Pa. At this state the mixture is two liquid phases.

Response

The example is the `data` part of the response; `units` and `meta` are as in section 2.

Field	Type	Unit	Description
`case`	string	–	`vapor-only`, `liquid-only (single liquid phase)`, `two-phase vapor-liquid (VL)`, `two-phase liquid-liquid (LL, aqueous+organic)` or `three-phase vapor-liquid-liquid (VLLE)`.
`V`, `L`	number	mole fraction	Vapor fraction and total liquid fraction.
`n_phases`	integer	count	Number of phases found.
`n_components_used`	integer	count	Components that took part (mole fraction above 0).
`vapor`, `aqueous`, `organic`	object or null	–	One object per phase, null if the phase does not exist: `beta` (phase fraction of the total, mole fraction), `MW` (g/mol), `Cp` (J/mol/K), `rho_mass` (kg/m³), `rho_molar_kmol_m3` (kmol/m³).
`mixture`	object	–	`MW` (g/mol), `Cp` (J/mol/K), `Cp_mass_kJ_kgK` (kJ/kg/K), `rho_mass` (kg/m³), `rho_molar_kmol_m3` (kmol/m³).


```
{
  "case": "two-phase liquid-liquid (LL, aqueous+organic)",
  "V": 0.0,
  "L": 1.0,
  "n_phases": 2,
  "n_components_used": 2,
  "vapor": null,
  "aqueous": {
    "beta": 0.4904076,
    "MW": 18.01528,
    "Cp": 81.16374,
    "rho_mass": 847.61,
    "rho_molar_kmol_m3": 47.0495
  },
  "organic": {
    "beta": 0.5095924,
    "MW": 84.89234,
    "Cp": 186.3908,
    "rho_mass": 663.8171,
    "rho_molar_kmol_m3": 7.819517
  },
  "mixture": {
    "MW": 52.09532,
    "Cp": 134.7867,
    "Cp_mass_kJ_kgK": 2.587308,
    "rho_mass": 689.1598,
    "rho_molar_kmol_m3": 13.22882
  }
}
```

Errors

`validation_error` (422), `unknown_component` (422), `property_unavailable` (422), `flash_failed` (422), `calculation_timeout` (504). Section 3 explains each code and what to do.

Notes

- The phase with the most water is reported as **aqueous**, the other liquid as **organic**.
- The first request for a new set of components builds the model and can take a few seconds (about 3 s for 25 components); later requests for the same set take milliseconds.

12. Enthalpy and heat duty

POST /eos_enthalpy_calc

http://192.168.11.216:4300/api/eos_enthalpy_calc

Enthalpy of one stream at an inlet and at an outlet condition, and the heat duty between them (outlet minus inlet), from a Peng-Robinson equation of state. A phase change between inlet and outlet, such as vaporisation, is included.

Request

Field	Type	Rules
`T_in`, `P_in`	number	Inlet temperature (K) and pressure (Pa), both above 0.
`T_out`, `P_out`	number	Outlet temperature (K) and pressure (Pa), both above 0.
`components`	array of text	Names or CAS numbers, no duplicates.
`zs`	array of numbers	Mole fractions or mole percent, same length and order. Rescaled to add up to 1.

```
{
  "T_in": 300,
  "P_in": 254920,
  "T_out": 368,
  "P_out": 254720,
  "components": [
    "74-82-8",
    "74-86-2",
    "74-85-1",
    "74-84-0",
    "74-99-7",
    "115-07-1",
    "74-98-6",
    "106-99-0",
    "106-98-9",
    "106-97-8"
  ],
  "zs": [0.02, 0.43, 60.6, 37.94, 0.01, 0.72, 0.14, 0.05, 0.05, 0.04]
}
```

Response

The example is the `data` part of the response; `units` and `meta` are as in section 2.

Field	Type	Unit	Description
`inlet`, `outlet`	object	–	State at that condition: `vapor_fraction` (mole fraction), `molecular_weight` (g/mol), `enthalpy_molar` (kJ/mol), `enthalpy_mass` (kJ/kg).
`duty.heat_duty_molar`	number	kJ/mol	Outlet minus inlet enthalpy. Positive = heat absorbed, negative = heat released.
`duty.sp_duty`	number	kJ/kg	Specific heat duty (per kilogram).
`n_components_used`	integer	count	Components that took part in the calculation.
`ethylene_ethane_kij_override`	boolean	–	true when ethylene and ethane are both present and the calibrated interaction parameter 0.015 was used for that pair.

```
{
  "inlet": {
    "vapor_fraction": 1.0,
    "molecular_weight": 28.97064,
    "enthalpy_molar": -0.04241087,
    "enthalpy_mass": -1.463926
  },
  "outlet": {
    "vapor_fraction": 1.0,
    "molecular_weight": 28.97064,
    "enthalpy_molar": 3.439142,
    "enthalpy_mass": 118.7113
  },
  "duty": { "heat_duty_molar": 3.481552, "sp_duty": 120.1752 },
  "n_components_used": 10,
  "ethylene_ethane_kij_override": true
}
```


Errors

`validation_error` (422), `unknown_component` (422), `property_unavailable` (422), `flash_failed` (422), `calculation_timeout` (504). Section 3 explains each code and what to do.

- `flash_failed`: The equilibrium calculation did not work at the inlet or at the outlet condition; the message says which.

Notes

- The example is a C2 stream (given in mole percent, so `meta.zs_normalized` is true) heated from 300 K to 368 K. Heating water from 300 K to 400 K at 101325 Pa gives a duty of 2717.545 kJ/kg, including the latent heat.
- Swapping inlet and outlet gives the same duty with the opposite sign.
- Components with a negligible mole fraction are skipped, so a long list with many zeros is fast. The first request for a new set of components builds the model (about 3 s).

13. Flow versus valve opening

POST /vent_flow_calc

http://192.168.11.216:4300/api/vent_flow_calc

Fits a polynomial to your measured flow versus valve opening (equivalent to Excel LINEST with Flow as y and Opening as x) and estimates the flow at further openings. No components, temperature or pressure are involved.

Request

Field	Type	Rules
`opening_values`	array of numbers	Valve opening of each data point, in %. At least 3 points.
`flow_values`	array of numbers	Flow at each opening, in any one unit. Same length as `opening_values`.
`degree`	integer	Requested polynomial degree, 1 to 10. Reduced automatically to what the points support (points − 2).
`estimate_at`	array of numbers	Openings (%) at which to estimate the flow. May be empty.
`clip_to_fit_range`	boolean	Default true: an opening outside the fitted range is evaluated at the nearest end of the range (still flagged as extrapolated).
`min_flow`	number	Default 0. Floor applied to every estimated flow.

```
{
  "opening_values": [19.7, 49.6, 89.7],
  "flow_values": [48601, 122050, 220698],
  "degree": 3,
  "estimate_at": [5, 50, 110],
  "clip_to_fit_range": true,
  "min_flow": 0.0
}
```

Response

The example is the `data` part of the response; `units` and `meta` are as in section 2.

Field	Type	Unit	Description
`fit.equation`	string	–	The fitted equation as text.
`fit.terms`	array	–	One entry per power, highest power first: `power`, `coefficient`, `standard_error`.
`fit.r_squared`, `fit.standard_error_y`, `fit.f_statistic`	number or null	–	Fit statistics. null when undefined (all flows equal, or an exact fit).
`fit.degree_requested`, `degree_used`, `degree_capped`	integer, integer, boolean	–	The degree you asked for, the degree fitted, and whether it was reduced.
`fit.n_points`, `df_residual`, `ss_regression`, `ss_residual`	number	–	Number of points, residual degrees of freedom, sums of squares.
`fit.opening_range`	object	%	`min` and `max` opening the curve was fitted on.
`estimates[]`	array	–	Per requested opening: `opening`, `opening_used`, `flow`, `extrapolated` (outside the fitted range), `floor_applied`.
`warnings[]`	array of text	–	Plain-language notes, for example that the degree was reduced.

```
{
  "fit": {
    "degree_requested": 3,
    "degree_used": 1,
    "degree_capped": true,
    "n_points": 3,
    "equation": "Flow(Opening) = +2458.61*Opening +143.197",
    "terms": [
      { "power": 1, "coefficient": 2458.613, "standard_error": 0.9992376 },
      { "power": 0, "coefficient": 143.197, "standard_error": 60.21554 }
    ],
    "r_squared": 0.9999998,
    "standard_error_y": 49.63446,
    "f_statistic": 6054004.0,
    "df_residual": 1,
    "ss_regression": 14914520000.0,
    "ss_residual": 2463.579,
    "opening_range": { "min": 19.7, "max": 89.7 }
  },
  "estimates": [
    {
      "opening": 5.0,
      "opening_used": 19.7,
      "flow": 48577.87,
      "extrapolated": true,
      "floor_applied": false
    },
    {
      "opening": 50.0,
      "opening_used": 50.0,
      "flow": 123073.8,
      "extrapolated": false,
      "floor_applied": false
    },
    {
      "opening": 110.0,
      "opening_used": 89.7,
      "flow": 220680.8,
      "extrapolated": true,
      "floor_applied": false
    }
  ],
  "warnings": [
    "The requested degree 3 was reduced to 1: 3 data points support at most degree 1.",
    "2 of 3 estimate openings lie outside the fitted range 19.7 to 89.7 % and were clamped to the fitted
range."
  ]
}
```


Errors

`validation_error` (422), `fit_failed` (422). Section 3 explains each code and what to do.

- `validation_error`: A list is shorter than 3 points, the two lists differ in length, the degree is outside 1 to 10, or a value is not a finite number.
- `fit_failed`: The data cannot be fitted: fewer different opening values than the degree needs. `error.details` gives the numbers.


```
{
  "status": "error",
  "error": {
    "code": "fit_failed",
    "message": "A degree-2 fit needs at least 3 different opening values; the data contain only 1.",
    "details": { "distinct_opening_values": 1, "required": 3, "degree_used": 2 }
  },
  "request_id": "a1b2c3d4e5f6"
}
```


Notes

- Flow, `standard_error_y` and the coefficients are in the unit of the flow values you send; nothing is converted. `meta` has only `request_id` and `elapsed_ms`.
- Use the lowest degree that describes the data. A high degree on few points follows the noise.

SYSTEM

14. Health check

GET /health

http://192.168.11.216:4300/api/health

Confirms that the service is running and tells you which version is deployed. It needs no body and no authentication, and is suitable for uptime monitoring.

Response

Field	Type	Description
`status`	text	`ok` while the service is alive.
`version`	text	The deployed API version.

```
{ "status": "ok", "version": "2.3.0" }
```

Errors

None. If the service is down, the request fails at the network level or returns a 5xx from the web server.

15. Client examples and version history

Postman

- Method **POST**, URL `http://192.168.11.216:4300/api/eos_stream_calc` (or any other endpoint).
- Body → **raw** → **JSON**, paste a request from the endpoint section, press **Send**. No API key is needed.

curl (Windows Command Prompt)

```
curl.exe -X POST http://192.168.11.216:4300/api/vf_calc -H "Content-Type: application/json" ^
  -d "{\"T\":300,\"P\":101325,\"components\":[\"water\",\"hexane\"],\"zs\":[0.5,0.5]}"
```

Python

```
import requests

BASE = "http://192.168.11.216:4300/api"
payload = {"T": 300, "P": 101325, "components": ["water", "hexane"], "zs": [0.5, 0.5]}

resp = requests.post(f"{BASE}/eos_stream_calc", json=payload, timeout=60)
body = resp.json()
if resp.ok:                           # HTTP 200
    print(body["data"]["case"], body["data"]["mixture"]["Cp"])
else:                                 # HTTP 4xx / 5xx
    err = body["error"]
    print(err["code"], err["message"], "request:", body["request_id"])
```

Version history

Version	Change
2.3.0	Added the Vent API (flow versus valve opening).
2.2.0	Added the Enthalpy API (enthalpy at inlet and outlet, heat duty).
2.1.0	Added the EOS Stream API. Documentation grouped by API.
2.0.0	Consistent response format with units, proper HTTP status codes and error codes, strict validation, null for absent phases.